

Final Report

Course: ENEL 300 – Winter 2026

Project: Autonomous RC Car

Group Number: 2

Team Members:

- Divy Tiwari
- Rodrigo Hernandez
- Isabella Huber
- Mai Le
- Atharva Mohandas

Instructor: Phillippe Gray

Contents

Abstract	2
Background	3
Electrical Architecture	3
Electrical Block Diagram:	3
Circuits.....	3
Metal Detector	4
Distance Sensing	4
Motor Drive Circuit	5
Power Management.....	5
Printed Circuit Board Design	6
Software Design.....	7
Architecture Diagrams	7
Driver Diagram	7
Receiver Diagrams	8
Code for Driver microcontroller.....	9
ESP-NOW Communication Functions	9
LCD and I2C Functions	9
app_main	10
Code for Receiver Microcontroller	10
Control and Mapping Functions.....	10
Servo Control Functions	11
Motor Control Functions	11
Ultrasonic Distance Sensor Functions	11
Mechanical Design.....	11
Rack and Pinion Steering	11
Chassis	12
Differential system	12
Car Floor.....	12
Metal Detection.....	12
Car Enclosure	12
Steering Wheel	13
Buttons Enclosure	13

Tests and Limitations	13
References	13
Appendix.....	13
Appendix A – Bill of Materials (BOM)	13
Appendix B – Code Pictures	14
Appendix C – 3D CAD Photos	28

Abstract

When first prompted to build an RC car, the first question was: Aren't most remote-controlled cars the same? RC cars in general are controlled by two motors in forward or rear wheel driving, with a few buttons or two joysticks to control the movement. For this project, the group's objective was to build something different; a more realistic car resembling the ones used in day-to-day life.

With a clear goal in mind, main characteristics of a car were added to the design: remote-controlled steering wheel, a differential rear wheel driving system, and a rack and pinion steering system. This would ensure that only one motor is always in use and that the driving experience would be as realistic as possible.

Background

Autonomous and remotely controlled vehicles are regularly used in robotics, industrial inspection, monitoring the environment and finally search and rescue. These systems combine technologies that allow the vehicle to navigate its environment from external sensing through the use of embedded control systems and mechanical structures.

The initial inspiration for this project was drawn from Formula One racing vehicles, which are known for their high-performance engineering, aerodynamic efficiency, and precision control systems. Due to this, our initial design was to create a functioning Formula One concept car and therefore use the name "Formula 2" for our group.

However, we realized we need a more multipurpose design, rather than purely a racing style. Our design pivoted to one inspired by classic luxury off-road vehicles, some of the original cars seen in the 30s-40s. This change in design made the car more capable for the specific tasks this RC car needs to meet, as well as combining modern control technologies within a design that visually reflects a vintage luxury off-road vehicle.

Electrical Architecture

Electrical Block Diagram:

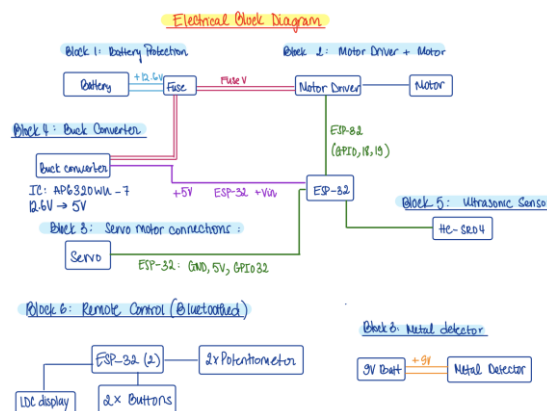


Figure 0: Electrical Block Diagram

Circuits:

Table 1: Electrical Components Used

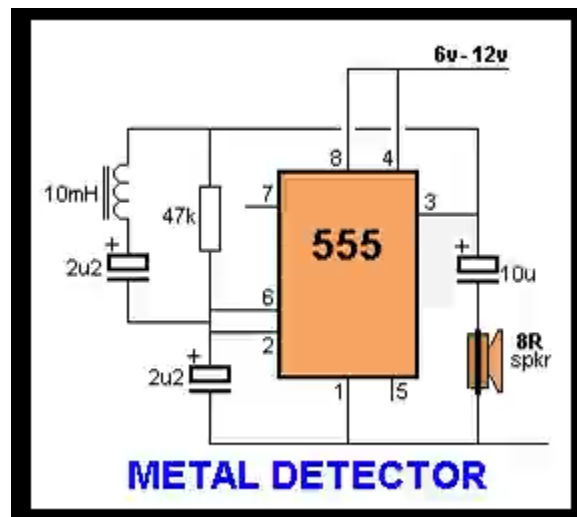
ESP-32 x 2
Servo Motor
DC Motor
Ultrasound Sensor
Potentiometer x 2
Buttons x 2
LCD Display

Metal Detector

Initially, the goal was to have a copper coil as the inductor that could detect metal by reacting to the change in frequency of oscillation caused by nearby magnetic materials. A circuit using the NE555R IC in Figure 1 was used for this process. However, through multiple trials, the circuit was unreliable, and it could only produce a slight change in buzzer vibration frequency. Potentiometers were used as an attempt to control the buzzer's vibration, and transistors were implemented; however, the results remained the same.

To deal with this problem, we decided to use continuity. By cutting the battery connection wire of the previous circuit, we essentially created an open circuit with two frayed wires hanging from the bottom of the car that are supported by two 3D-printed support legs. When our cables encounter a metal plate on the ground, the circuit would then become a closed circuit, allowing the buzzer to sound only when there is metal underneath the car.

Figure 1: Metal Detector Circuit Diagram



Distance Sensing

To reliably measure the distance of objects in front of the car, an ultrasonic sensor (HC – SR04) mounted on the front of the car is used to perform this task. This sensor provides us the measurements using a sound

wave to measure the distance it takes for the emitted pulse to bounce off the object and get received by the echo. The following table shows the pins used:

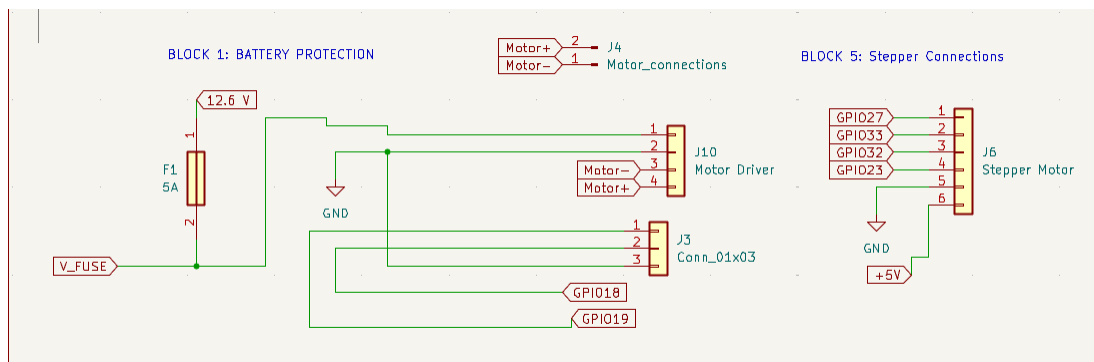
Table 1: Ultrasonic sensor pins

GND	GND
VCC	GPIO 25
ECHO	GPIO 26
TRIG	+5V

The reason for choosing the HC-SR04 is due to its reliability and abundance as the team members are all familiar with the component and how it works. Furthermore, the distance between the car and the object in front is then displayed on the LDC display.

Motor Drive Circuit

Figure 2: Motor Drive Circuit Kicad Schematic



- This circuit is the heart of the remote-controlled car movements. Figure 2 shows all the GPIO pin, battery, and motor connections of the driver through 2 separate pin headers (J3, J10).
- The servo motor is used to control the motor angle of rotation that receives signals from the remote commands connected by Bluetooth so that the user can manually control the movement and direction of the car while driving it.
- Initially, a stepper motor was supposed to control the car's rotation, but it was then replaced by a servo motor, leaving a few pins on the PCB unused.
- Servo Pin Configurations:

GND	ESP-32 GND
VCC	ESP-32 +5V
PWM	ESP-32 GPIO 32

Power Management

Voltage Levels & Battery Rationale:

This RC car requires power for 2 key aspects that will ensure complete functionality, and a big part of this is to provide a brief justification for all the chosen voltage levels, so that it is easy to see how said choices benefit our system and contribute to our design.

1: Drive System: The drive system in this car includes a 12V 200RPM DC and a 12.6V Li-Ion Rechargeable Battery

- The motor was chosen to be 12V for 2 reasons. Firstly, a higher voltage means a lower current draw, while keeping the power constant. This ensures efficiency as well as makes sure that under load (climbing an incline), the motor will not draw too much current. Secondly, 12V motors are quite common in RC Car applications, so our group wanted to go with something that was tried and tested, so that we would limit the window for failure.
- The battery was chosen to be at 12V primarily for the motor as it needs to deliver sufficient voltage for it to run properly. However, it is also important to note that the chemistry of this battery was also considered. When compared Li-Ion cells with other kinds such as lead batteries or Ni-MH batteries, it is found that Li-Ion cells exhibit much longer runtimes because lithium atoms hold a larger amount of energy during electrochemical reactions. This also means that the battery is lighter because a large number of cells are not needed.

2: Control System: The control system mainly consists of the ESP-32 Module, which controls a variety of components on the car

- The ESP-32 Module needs a supply of 5V and the reason this module was chosen is because it is easy to work with when compared to a lot of other microcontrollers. This is powered through a buck converter discussed in a separate section.

Motor Power Draw & Protection Circuitry:

Power Calculations

To properly show how the motor draws power, it is necessary to show how much current the motor will draw under certain conditions. These conditions, along with the current draw for each condition, is supported by a calculation below:

- Condition 1: No-Load Current (motor is freely spinning on its own):
 - o To calculate no load current, we need to utilize the following formula
 - $I = \frac{V - V_b}{R}$, where V = supply voltage, V_b = Back EMF(which is very close to supply value at no load) and R = 1 (internal winding resistance)
 - $I = \frac{12 - 11.3}{1} = 0.7A$
- Condition 2: Load Current (Primarily while climbing the 20-degree incline):
 - o To approximate the current while climbing the incline, it is necessary to know the weight of the car as this determines exactly how much force the motor needs to overcome to climb the ramp. This will also determine the torque needed and therefore determine the current needs. It is also necessary to know the radius of the wheels to convert force into torque.
 - First, we will find the Force using an approximate vehicle weight and the incline angle of the ramp.
 - $F = mgsin(\theta)$, $m = 0.9kg$, $\theta = 20$

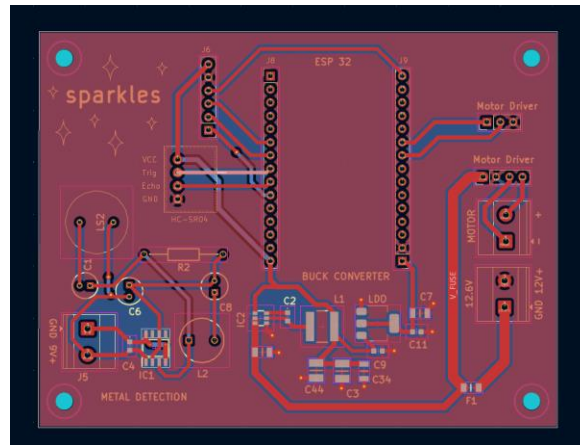
- $F = (0.9kg) \left(\frac{9.81m}{s^2} \right) \sin(20) = 3.05N$
- Next, using the wheel radius, we will convert that force into torque
- $T = Fr$, $F = 3.05N$, $r(\text{wheel radius}) = 1.5in = 0.038m$
 - $T = (3.05N)(0.038m) = 0.116Nm$
- Now, we need to make use of the stall current to estimate the load current This is because stall current is the maximum current a DC motor can draw so we can say that the load current is simply a fraction of the maximum current. However, in order to make an accurate estimate, we also need to use the torque calculated in the previous step.
- $I_{load} = I_{stall} \left(\frac{T_{load}}{T_{stall}} \right)$, we assume that the stall torque is exactly 1Nm as small, geared DC motors usually have manufacture specified values ranging from 0.5Nm – 2Nm, so 1Nm is an average estimate.
 - $I_{load} = 12.6A \left(\frac{0.116Nm}{1Nm} \right) = 1.39A$
- Condition 3: Stall Current (If the motor stalls, how much current will it draw)
 - At a stall, the rotor is stationary, so the motor isn't spinning. This means the back EMF is 0 and therefore, the current is only limited by the winding resistance. So, we can find the stall current by doing a simple Ohm's Law Calculation
 - $I = \frac{12.6V}{1\Omega} = 12.6A$

Protection Circuitry

As briefly mentioned above, our RC car also employed protection circuitry that was vital in case of a fault. The protection element that was chosen was a fuse rated for exactly 5A. This value was chosen due to the current supply limitations of our battery, which could only output 5A of current. If it attempted to supply more, the BMS(Battery Management System) inside the battery would instantly cause it to trip. This would be catastrophic as our entire system would lose power. Now, the only electrical component in our circuit that would ever draw such a high level of current is the motor, in case it stalled. So, in reference to Fig 0. , our solution was to have two lines running in parallel from our battery. One would go towards the driving system in series with the aforementioned fuse, and the other would go towards the control system. This means that in the case of a motor stall, the fuse ensures that only the connection to the motor is cut off and hence, our electronic components would stay powered. This also protects the wiring from carrying levels of current way higher than what it is rated for.

Printed Circuit Board Design

Figure 3: Custom PCB Design



1. Functions integrated on each board:

- Almost all of the car's electrical systems are included in the custom PCB including battery protection using a fuse, motor driver and motor connections, buck converter circuit using an IC, ESP-32, ultrasonic sensor, and metal detection circuit.

2. Connector interfaces and their purpose:

- Three screw terminals are used for battery and motor driver connections.
- Multiple headers are used to connect and mount the microcontroller (ESP-32), ultrasonic sensor (HS-SR04), servo motor.
- Buck converter IC input pin is connected to the +12V battery input and outputs +5V to power the microcontroller.
- The +5V buck converter output is also connected to our LDO that provides the 3.3V output voltage for the headlights of the car.

3. Thermal Design Considerations

- The buck converter circuit has its components placed as close together as possible to improve efficiency and prevent too much internal heating.
- Trace widths are chosen based on the current levels for each block. Current going from battery to fuse has the widest trace of 2.0 mm to reduce heat dissipation. Connection from fuse to buck converter ic is 1.4 mm. All other smaller power lines and logic are around 0.5 - 1.0 based on their positioning.
- 80x100 mm 2 layers board. After routing all the components, a copper ground filled zone was used to create the board's common ground. Ground vias were also used to ground buck converter ic , decoupling and filter capacitors to improve efficiency and create extra reliable paths to the ground plane.

4. Layout Decisions

- The ESP32 headers are placed in the middle for the purpose of having short routing distances between it and other components like the sensor, motor driver, and servo motor which are also positioned so that they are placed right next to the connected GPIO pins.

- Metal detector circuit is placed separately on the bottom left with its own battery connector and ground connections to prevent noise.

Software Design

From a software perspective two ESP-32 were used, the ESP-NOW protocol was chosen as the best option to connect both microcontrollers, protocol which connects specifically ESP-32 to work seamlessly together. In addition, for this sections there are two main design phases, one for the ESP-32 that goes in the car which is called the “receiver” and the second for the ESP-32 that is used as the remote control, which is called the “driver”.

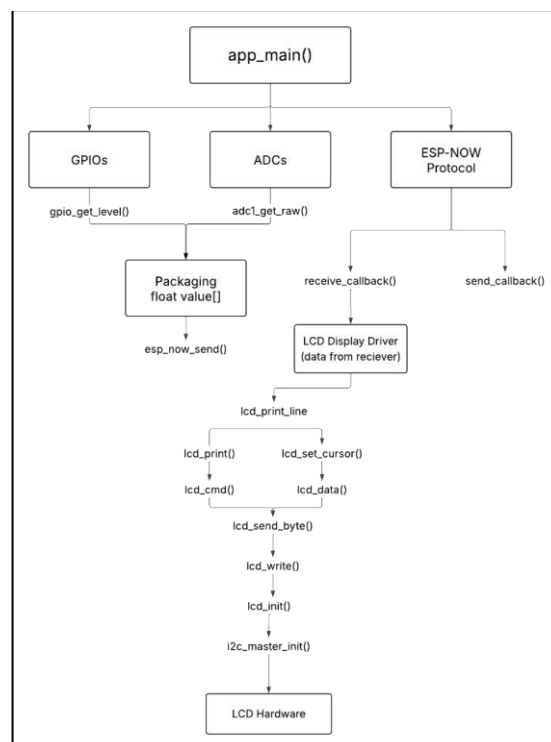
Architecture Diagrams

There are two separate architecture diagrams, one for the driver and the second for the receiver, these sections show both.

Driver Diagram

For the driver, the main objective is to control the LCD screen display, get the button and potentiometer inputs, and send the inputs to the receiver board. These main objectives are reflected in the following flow diagram for the driver:

Figure 3: Driver Architecture Diagram



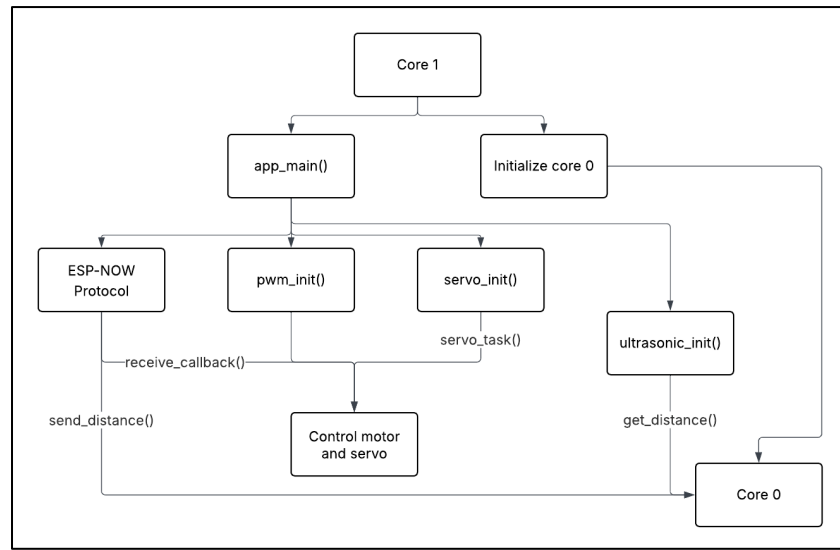
Built with LucidChart

It is important to note that receive callback is only executed when information is sent by the other ESP-32. This is how both diagrams are connected.

Receiver Diagrams

One of the great particularities of the ESP-32 is that it has two cores, which means that if a process takes too long to run and it needs to run something else immediately, instead of using interrupts, this code can instead be executed in the second core. In this case all the driving, receiving data and sending data are executed on the first core, while ultrasonic and servo usage on the second core. This is reflected in the following diagram:

Figure 4: Receiver Architecture Diagram



Code for Driver microcontroller

ESP-NOW Communication Functions

The remote control had to use 2 potentiometers, 2 buttons, and an LCD screen display. The potentiometers were used for the steering wheel and pedal, and buttons for driving forward and backwards. The code editor used was VS Code with the Espressif-IDE extension installed to connect to the ESP-32.

All the initial functions called were coded, alongside the “`app_main`” (See Figure 5 in Appendix B).

How each function was coded and what their role is in the program is explained below. The explanation is led by how these functions work before attending to “`app_main`”, to make it more understandable.

The function ‘`send_callback`’ is executed right after a call to the in-built ESP-NOW function “`esp_now_send`” which is executed within “`app_main`” (See Figure 6 in Appendix B). ‘`send_callback`’ will print out in the terminal the status of the data sent to the other ESP-32, if “Success” then data was sent properly and if “Fail” data was not able to be sent and most of the time this means that the ESP-32 are not connected to each other. The `receiver_mac` list is where the MAC address of the receiver ESP-32 is stored, and it is used to send data directly to it.

Likewise, `receive_callback` is called whenever data is received from the other ESP-32, and since the only point of data received from the car is the distance sensed by the ultrasound, only one float type data point is used here, which is then passed onto the LCD display (See Figure 7 in Appendix B). The volatile integer

type variable “flag” controls how often the LCD screen is updated, while the buffer controls the text printed out on the screen.

LCD and I2C Functions

For writing into the LCD screen, I2C communication is used. The function ‘lcd_write’ creates that I2C command (See Figure 8 in Appendix B). The code sends the starting condition for any writing done on the I2C bus. Writing is specified by shifting the 7-bit LCD address to the left by 1 bit and adding the specific write bit that is needed to tell the screen that writing is happening, which in this case is 0. The function ‘lcd_send_byte’ then sends a full byte to the LCD screen (See Figure 9 in Appendix B).

Along with that, ‘lcd_cmd’ is used to send a command to ‘lcd_send_byte’, where the number 0 implies that the type of data sent is a command, and this is used later to offset the text written onto the LCD screen (See Figure 10 in Appendix B). Likewise, ‘lcd_data’ is used in combination with ‘lcd_send_byte’, however in this case it is written instead of sending a type of command (See Figure 11 in Appendix B). With ‘lcd_init’, the LCD display is turned on, and all its pins are initialized, following the display data sheet (See Figure 12 in Appendix B).

The function ‘lcd_set_cursor’ simply sets where on the LCD display the text appears, where upper-left was chosen (See Figure 13 in Appendix B). In addition, ‘i2c_master_init’ initializes the I2C pins used from the ESP-32, I2C_MASTER_SDA_IO and I2C_MASTER_SCL_IO, which are specified in the program definitions (See Figure 14 in Appendix B).

After all that setup needed for the LCD display, ‘lcd_print’ allows any text to be printed on the LCD display (See Figure 15 in Appendix B), while ‘lcd_print_line’ combines ‘lcd_print’ with ‘lcd_set_cursor’ to appropriately insert the text wherever it was chosen (See Figure 16 in Appendix B).

app_main

This section is divided into multiple figures, since the “app_main” function is the longest. The first portion describes the initialization of the ESP_NOW protocol between the two ESP-32, as well as the LCD screen initialization function. (See Figure 17 in Appendix B)

In broad terms the function turns on the Wi-fi capabilities of the ESP-32, sets the channels and connects a “peer” to itself, this peer is the MAC address of the receiver ESP-32. Two if statements to ensure proper connection were used, one to ensure that the code wouldn’t execute without proper Wi-fi initialization and the other to ensure the receiver ESP-32 was properly paired. Next the function ‘receive_callback’ is added as the function to be executed when data is received from the other ESP-32. (See Figure 18 in Appendix B)

This second portion of app_main initializes the two ADC channels and two GPIO pins used by the program for the potentiometers and buttons. It then pulls raw data from those pins and saves it into a list that will then be sent to the receiver ESP-32. (See Figure 19 in Appendix B)

This final section of app_main implements the logic used to send data to the receiver. The code takes ‘n’ points of data for both potentiometers and averages them to provide a more accurate reading. Finally, it sends all the data needed to the receiver.

Code for Receiver Microcontroller

The receiver microcontroller oversaw controlling every aspect of the car. Which includes the ultrasonic sensor, the motor driver, and the servo used for steering. All the predefined functions used for the code are shown in Figure 20. (See Figure 20 in Appendix B)

Control and Mapping Functions

The function ‘receive_callback’ is the main control of the servo and motor driver. It first receives the data from the driver and gives instructions to the driver to go forwards or backwards depending on the press of the buttons, while also adjusting the servo angle depending on the potentiometer value read. This value is then read by ‘servo_task’, which is executed independently (See Figure 21 in Appendix B). To support this, map takes a boundary of values and makes them proportional to another boundary of values (See Figure 22 in Appendix B).

Servo Control Functions

For the steering system, ‘servo_init’ initializes the servo by configuring all its needed characteristics (See Figure 23 in Appendix B). Since servos work by sending a determined duty cycle to set the angle, ‘servo_angle_to_duty’ takes an angle and returns the duty cycle needed to achieve that specified angle (See Figure 24 in Appendix B). Then ‘servo_set_angle’ compiles all that is needed to set the correct angle to the servo (See Figure 25 in Appendix B). The function ‘servo_task’ is continuously running in core 1, with no interrupts needed, and it uses a filter to make the servo work smoothly while also specifying a deadzone in which the servo will not move to account for small voltage changes and ensure the servo is as precise as possible (See Figure 26 in Appendix B).

Motor Control Functions

For motor control, ‘pwm_init’ initializes the PWM pulses needed to control the motor driver (See Figure 27 in Appendix B). The functions ‘motor_forward()’, ‘motor_reverse()’ and ‘motor_stop()’ then send the correct PWM pulses to the motor driver to move the car forwards, backwards or to stop it (See Figure 28 in Appendix B).

Ultrasonic Distance Sensor Functions

For the ultrasonic subsystem, ‘ultrasonic_init’ initializes the ultrasonic sensor (See Figure 29 in Appendix B). The function ‘get_distance’ takes the distance with the ultrasonic sensor three times, and if any of them are successful then the distance is returned. Various tests had to be done to ensure this function would work, especially in the absence of a regulator in the final PCB design (See Figure 30 in Appendix B).

Then ‘send_distance’ sends the distance constantly to the driver to be displayed on the LCD screen (See Figure 31 in Appendix B). Finally, ‘ultrasonic_task’ is constantly run in core 0, and a similar smoothing function is used for the results of the ultrasound as the one used for the servo (See Figure 32 in Appendix B).

Mechanical Design

The mechanical design of the car involves a differential system, a rack and pinion steering system, the chassis, and the enclosure of the car. Each one of these systems, especially the gearing, was designed to fit seamlessly together, and tolerances were polished in SolidWorks assemblies to ensure fittings weren’t off.

On the other hand, the remote control had a design phase that involved building the steering wheel and its casing. While also building a small casing for the forwards and backwards buttons.

Rack and Pinion Steering

The rack and pinion steering system was designed to hold the two front wheels, four small 11mm ball bearings were used for smoothly steering the system, while two bigger 23mm ball bearings were used for rotating the wheels. (See Figure 33 in Appendix C).

The mounting was simple, since tolerances were accounted for in advance. The chassis has the holder for two of the four small ball bearings used, which is how these two connect. The top side view of how it connects to the chassis is shown in the following figure. (See Figure 34 in Appendix C).

Chassis

The chassis of the vehicle was designed to be very flexible in terms of what we could build next. The main goal was to always be capable of adapting to a system, failing to print on time, or unexpectedly not working or fitting in the proper manner. (See Figure 35 in Appendix C).

The support for the two rear wheels and their differential system can be observed there too. The rear wheels are supported by 4 ball bearings which ensure that the differential stays in place properly and runs smoothly. This was accounted for after the first sprint when the differential system was completely touching the ground as the car ran. (See Figure 36 in Appendix C).

Differential system

The differential system was the most complex design decision the team made. It involved making a crown or ring gear, which would drive the two differential gears in parallel, these would make the side gears have different speeds which is the purpose of this system. After trial and error the final design for the gears was the following figure. (See Figure 37 in Appendix C).

A casing was then designed for the crown gear. (See Figure 38 in Appendix C).

Once mounted, the system would look as follows. (See Figure 39 in Appendix C).

Car Floor

This is the piece that holds the motor, the battery, the PCB, and the servo (this last needs and additional part which will be described later). The following design was chosen to account for mass distribution and large moments of inertia that could make the car tilt backwards as it goes up the ramp. (See Figure 40 in Appendix C).

The holes observed in the design were used to screw other pieces in place as well as mount it in the chassis. However, this model is thin enough to easily be drilled in case other pieces need to be mounted, which became an important design decision. The PCB goes on a separate upper piece, which can be observed in the fully assembled design. The servo is then mounted with a small piece attached to this same system which can also be observed in that figure. (See Figure 41 in Appendix C).

Metal Detection

The small system that holds the cables for metal detection is shown in the figure. (See Figure 42 in Appendix C).

Car Enclosure

The car enclosure, the most visible part, is separated into two pieces: the front of the car, which holds the headlights and back of the car, which encloses the circuitry and holds the switches for turning different systems on or off. Another piece above will act as a cover of the PCB and wiring and include hinge joints for access to components while still providing shelter and shape. (See Figure 43 in Appendix C).

These two parts were designed to be screwed together in place and resemble an old generation car design. (See Figures 44 and 45 in Appendix C).

Steering Wheel

The steering wheel used to control the car had to be designed to use a potentiometer for measuring steering angles. Therefore, an enclosure for the steering wheel was made too. The full assembly can be observed in the following figure. (See Figure 46 in Appendix C).

Buttons Enclosure

The two buttons that drive the car forwards or backwards were enclosed in the following manner. (See Figure 48 in Appendix C).

The piece that holds the potentiometer is shown in the following section view of the assembly. (See Figure 47 in Appendix C).

Additionally, a bottom piece is needed which goes attached to the above one through screws. (See Figure 49 in Appendix C).

Finally, the piece that holds it together. (See Figure 50 in Appendix C).

Tests and Limitations

The mechanical performance was validated through testing on flat ground and on incline. Our car showed smooth motion and successfully climbed a $\sim 21^\circ$ incline, confirming that we have sufficient torque from the drivetrain. However, we observed reduced speed under load when the car climbed the ramp as well as minor steering instability. This is likely due to the torque constraints, to combat this, future revisions will include testing with different motors that takes higher current and voltage to run faster and to fix the steering going up the ramp, we use a different servo motor or a motor with a different mechanism.

References

- [1]“HD44780U (LCD-II) (Dot Matrix Liquid Crystal Display Controller/Driver).” Hitachi, 1999.
- [2]“Large current 50A h bridge driver module, MOSFET, PWM isolation, motor forward reverse, High Power Single Channel Motor Driver Module, 3.3V-12V power, for Motor Control,” *Amazon*. [Online]. Available: <https://a.co/d/b1ZrjJq>. [Accessed: 13-Feb-2026]
- [3]“DMiotech micro DC 12V 200RPM geared motor with bracket gear reduction motors 37mm diameter gearbox micro speed reduction for RC car, Robot Model, DIY Toys,” *Amazon*. [Online]. Available: <https://a.co/d/apkxDoR>. [Accessed: 13-Feb-2026]

Appendix

Appendix A – Bill of Materials (BOM)

Component	Description	Quantity	Cost
Microcontroller Unit	ESP-32	2	\$7.32
HC-SR04	Ultrasonic Distance Sensor	1	Provided
LCD Display	Display for Distance	1	Provided via Makerspace
Motor Driver	Motor driver for DC motors	1	\$9.69
DC Motor	Gear motors for propulsion		\$27.49
RC Car Battery	Main power supply for motors	1	\$29.99
Detection Battery	Power supply for metal detection circuit	1	Free
PCB Components	Resistors, capacitors, connectors	Various	\$21.25
Wheels	Wheels for the RC car	4	\$16.49
Ball Bearings		Whole Pack	\$8.49

Appendix B – Code Pictures

Figure 5: Function definitions of the Driver


```

void send_callback(const esp_now_send_info_t *tx_info, esp_now_send_status_t status);
void receive_callback(const esp_now_recv_info_t *recv_info, const uint8_t *data, int len);
void lcd_send_byte(uint8_t data, uint8_t mode);
void lcd_cmd(uint8_t cmd);
void lcd_data(uint8_t data);
void lcd_init();
void lcd_set_cursor(int col, int row);
void lcd_print(const char *text);
void i2c_master_init();
void lcd_print_line(int row, const char *text);
void lcd_write(uint8_t data);

```

Figure 6: Function send_callback

```

void send_callback(const esp_now_send_info_t *tx_info, esp_now_send_status_t status) {
    ESP_LOGI(TAG, "Send status: %s",
             status == ESP_NOW_SEND_SUCCESS ? "Success" : "Fail");

    ESP_LOGI("SENDER", "Sent to: %02X:%02X:%02X:%02X:%02X:%02X",
             receiver_mac[0],
             receiver_mac[1],
             receiver_mac[2],
             receiver_mac[3],
             receiver_mac[4],
             receiver_mac[5]);
}

```

Figure 7: Function receive_callback

```

void receive_callback(const esp_now_recv_info_t *recv_info, const uint8_t *data, int len) {
    if (len == sizeof(float) && flag == 5) {
        float distance;
        flag = 0;
        memcpy(&distance, data, sizeof(distance));
        char buffer[21];
        snprintf(buffer, sizeof(buffer), "Distance: %-7.3f  ", distance);
        lcd_print_line(0, buffer);
    } else {
        flag++;
    }
}

```

Figure 8: Function lcd_write

```

void lcd_write(uint8_t data)
{
    i2c_cmd_handle_t cmd = i2c_cmd_link_create();

    i2c_master_start(cmd);
    i2c_master_write_byte(cmd, (LCD_ADDR << 1) | I2C_MASTER_WRITE, true);
    i2c_master_write_byte(cmd, data, true);
    i2c_master_stop(cmd);

    i2c_master_cmd_begin(I2C_MASTER_NUM, cmd, pdMS_TO_TICKS(50));
    i2c_cmd_link_delete(cmd);
    vTaskDelay(pdMS_TO_TICKS(1));
}

```

Figure 9: Function `lcd_send_byte`

```

void lcd_send_byte(uint8_t data, uint8_t mode)
{
    uint8_t high = data & 0xF0;
    uint8_t low  = (data << 4) & 0xF0;

    uint8_t control = LCD_BACKLIGHT;
    if (mode) control |= RS;

    // HIGH
    lcd_write(high | control);
    lcd_write(high | control | ENABLE);
    lcd_write(high | control);

    // LOW
    lcd_write(low | control);
    lcd_write(low | control | ENABLE);
    lcd_write(low | control);

    vTaskDelay(pdMS_TO_TICKS(2));
}

```

Figure 10: Function `lcd_cmd`

```

void lcd_data(uint8_t cmd)
{
    lcd_send_byte(cmd, 0);
    vTaskDelay(pdMS_TO_TICKS(2));
}

```

Figure 11: Function `lcd_data`

```

void lcd_data(uint8_t data)
{
    lcd_send_byte(data, RS);
    vTaskDelay(pdMS_TO_TICKS(1));
}

```

Figure 12: Function `lcd_init`

```

void lcd_init()
{
    vTaskDelay(pdMS_TO_TICKS(50));

    lcd_send_byte(0x33, 0);
    lcd_send_byte(0x32, 0);
    lcd_send_byte(0x28, 0); // 4-bit, 2 line
    lcd_send_byte(0x0C, 0); // display on
    lcd_send_byte(0x06, 0); // entry mode
    lcd_send_byte(0x01, 0); // clear

    vTaskDelay(pdMS_TO_TICKS(5));
}

```

Figure 13: Function `lcd_set_cursor`

```

void lcd_set_cursor(int col, int row)
{
    int row_offsets[] = {0x00, 0x40};
    lcd_cmd(0x80 | (col + row_offsets[row]));
}

```

Figure 14: Function `i2c_master_init`

```

void i2c_master_init()
{
    i2c_config_t conf = {
        .mode = I2C_MODE_MASTER,
        .sda_io_num = I2C_MASTER_SDA_IO,
        .scl_io_num = I2C_MASTER_SCL_IO,
        .sda_pullup_en = GPIO_PULLUP_ENABLE,
        .scl_pullup_en = GPIO_PULLUP_ENABLE,
        .master.clk_speed = I2C_MASTER_FREQ_HZ
    };

    i2c_param_config(I2C_MASTER_NUM, &conf);
    i2c_driver_install(I2C_MASTER_NUM, conf.mode, 0, 0, 0);
}

```

Figure 15: Function lcd_print

```

void lcd_print(const char *text)
{
    while (*text)
    {
        lcd_data(*text++);
    }
}

```

Figure 16: Function lcd_print_line

```

void lcd_print_line(int row, const char *text)
{
    lcd_set_cursor(0, row);
    lcd_print(text);
}

```

Figure 17: First portion of app_main

```

void app_main(void) {
    i2c_master_init();
    lcd_init();
    nvs_flash_init();
    esp_netif_init();
    esp_event_loop_create_default();

    printf("Scanning I2C...\n");

    wifi_init_config_t cfg = WIFI_INIT_CONFIG_DEFAULT();
    esp_wifi_init(&cfg);
    esp_wifi_set_mode(WIFI_MODE_STA);
    esp_wifi_start();
    esp_wifi_set_channel(1, WIFI_SECOND_CHAN_NONE);

    if (esp_now_init() != ESP_OK) {
        ESP_LOGE(TAG, "ESP-NOW init failed");
        return;
    }

    esp_now_register_send_cb(send_callback);

    esp_now_peer_info_t peer = {0};
    memcpy(peer.peer_addr, receiver_mac, 6);
    peer.channel = 1;
    peer.encrypt = false;

    if (esp_now_add_peer(&peer) != ESP_OK) {
        ESP_LOGE(TAG, "Failed to add peer");
        return;
    }

    esp_now_register_recv_cb(receive_callback);
}

```

Figure 18: Second portion of app_main

```

adc1_config_width(ADC_WIDTH_BIT_12);
adc1_config_channel_atten(ADC1_CHANNEL_6, ADC_ATTEN_DB_11);
adc1_config_channel_atten(ADC1_CHANNEL_5, ADC_ATTEN_DB_11);

// Configure as input
gpio_set_direction(BUTTON1_GPIO, GPIO_MODE_INPUT);
gpio_set_direction(BUTTON2_GPIO, GPIO_MODE_INPUT);

// Enable internal pull-ups
gpio_set_pull_mode(BUTTON1_GPIO, GPIO_PULLUP_ONLY);
gpio_set_pull_mode(BUTTON2_GPIO, GPIO_PULLUP_ONLY);

float adc_value = adc1_get_raw(ADC1_CHANNEL_6);
float adc_value2 = adc1_get_raw(ADC1_CHANNEL_5);
float current_b1 = gpio_get_level(BUTTON1_GPIO);
float current_b2 = gpio_get_level(BUTTON2_GPIO);

float value[] = {adc_value, current_b1, current_b2, adc_value2};

```

Figure 19: Third portion of app_main

```

while (1) {
    printf("Pot1: %f | Pot2: %f\n", adc_value, adc_value2);

    esp_err_t result = esp_now_send(receiver_mac, (uint8_t *)&value, sizeof(value));

    if (result != ESP_OK) {
        ESP_LOGE(TAG, "Send error: %d", result);
    }

    ESP_LOGI(TAG, "Sent: %d", value);

    int n = 10;
    adc_value = 0;
    adc_value2 = 0;
    for (int i = 0; i < n; i++) {
        adc_value += adc1_get_raw(ADC1_CHANNEL_6);
        adc_value2 += adc1_get_raw(ADC1_CHANNEL_5);
        vTaskDelay(pdMS_TO_TICKS(10));
    }
    adc_value /= n;
    adc_value2 /= n;
    current_b1 = gpio_get_level(BUTTON1_GPIO);
    current_b2 = gpio_get_level(BUTTON2_GPIO);

    value[0] = adc_value;
    value[1] = current_b1;
    value[2] = current_b2;
    value[3] = adc_value2;

    vTaskDelay(pdMS_TO_TICKS(200 - (10 * n)));
}

```

Figure 20: Function definitions of the Receiver

```

void servo_init();
void servo_set_angle(int angle);
uint32_t servo_angle_to_duty(int angle);
void servo_task(void *arg);

void pwm_init();
void motor_forward(uint8_t speed);
void motor_reverse(uint8_t speed);
void motor_stop();

void ultrasonic_init();
float get_distance();

void send_distance(float distance);

void ultrasonic_task(void *arg);

void receive_callback(const esp_now_recv_info_t *recv_info, const uint8_t *data, int len);

long map(long x, long in_min, long in_max, long out_min, long out_max);

```

Figure 21: Function receive_callback()

```

void receive_callback(const esp_now_recv_info_t *recv_info, const uint8_t *data, int len) {
    float value[4];

    // Copying the values given by the other esp32 to the list "value"
    memcpy(value, data, sizeof(value));

    // Printing value 1, for debugging purposes
    ESP_LOGI(TAG, "Received: %f", value[1]);

    // Scaling potentiometer values to fit stepper steering
    target_angle = map(value[0], 0, 4095, 5, 175);

    printf("Values: %f\n", value[0]);

    // Running motor forwards or backwards depending on the button inputs (to be changed)
    if (value[1] == 0){
        motor_forward(255);
    } else if (value[2] == 0) {
        motor_reverse(255);
    } else {
        motor_stop();
    }

    // Printing MAC address of the esp32
    ESP_LOGI(TAG, "From MAC: %02X:%02X:%02X:%02X:%02X:%02X",
        recv_info->src_addr[0],
        recv_info->src_addr[1],
        recv_info->src_addr[2],
        recv_info->src_addr[3],
        recv_info->src_addr[4],
        recv_info->src_addr[5]);
}

```

Figure 22: Function map()

```

long map(long x, long in_min, long in_max, long out_min, long out_max) {
    return (x - in_min) * (out_max - out_min) / (in_max - in_min) + out_min;
}

```

Figure 23: Function servo_init()

```

void servo_init() {
    ledc_timer_config_t timer = {
        .speed_mode      = SERVO_MODE,
        .timer_num       = SERVO_TIMER,
        .duty_resolution = SERVO_RESOLUTION,
        .freq_hz         = SERVO_FREQ,
        .clk_cfg         = LEDC_AUTO_CLK
    };
    ledc_timer_config(&timer);

    ledc_channel_config_t channel = {
        .gpio_num  = SERVO_GPIO,
        .speed_mode = SERVO_MODE,
        .channel    = SERVO_CHANNEL,
        .timer_sel  = SERVO_TIMER,
        .duty       = 0,
        .hpoint     = 0
    };
    ledc_channel_config(&channel);
}

```

Figure 24: Function servo_angle_to_duty()

```

uint32_t servo_angle_to_duty(int angle) {
    // Clamp angle
    if(angle < 0) angle = 0;
    if(angle > 180) angle = 180;

    uint32_t pulse_width = SERVO_MIN_PULSEWIDTH +
        (angle * (SERVO_MAX_PULSEWIDTH - SERVO_MIN_PULSEWIDTH) / SERVO_MAX_DEGREE);

    // Convert µs → duty
    uint32_t duty = (pulse_width * (1 << SERVO_RESOLUTION)) / (1000000 / SERVO_FREQ);

    return duty;
}

```

Figure 25: Function servo_set_angle()


```

void servo_set_angle(int angle) {
    uint32_t duty = servo_angle_to_duty(angle);

    ledc_set_duty(SERVO_MODE, SERVO_CHANNEL, duty);
    ledc_update_duty(SERVO_MODE, SERVO_CHANNEL);
}

```

Figure 26: Function servo_task ()

```

void servo_task(void *arg)
{
    float filtered = 90;
    int current_angle = 90;

    while(1)
    {
        // Smooth input
        filtered = 0.8 * filtered + 0.2 * target_angle;
        int new_angle = (int)filtered;

        // Deadzone
        if(abs(new_angle - current_angle) > 2)
        {
            current_angle = new_angle;
            servo_set_angle(current_angle);
        }

        vTaskDelay(pdMS_TO_TICKS(20));
    }
}

```

Figure 27: Function pwm_init()

```

void pwm_init(void) {
    // Configure timer
    ledc_timer_config_t ledc_timer = {
        .speed_mode      = PWM_MODE,
        .duty_resolution = PWM_RESOLUTION,
        .timer_num       = PWM_TIMER,
        .freq_hz         = PWM_FREQ,
        .clk_cfg         = LEDC_AUTO_CLK
    };
    ledc_timer_config(&ledc_timer);

    // Configure channel for IN1
    ledc_channel_config_t ch1 = {
        .gpio_num        = IN1_GPIO,
        .speed_mode      = PWM_MODE,
        .channel         = PWM_CHANNEL_1,
        .timer_sel       = PWM_TIMER,
        .duty            = 0,
        .hpoint          = 0
    };
    ledc_channel_config(&ch1);

    // Configure channel for IN2
    ledc_channel_config_t ch2 = {
        .gpio_num        = IN2_GPIO,
        .speed_mode      = PWM_MODE,
        .channel         = PWM_CHANNEL_2,
        .timer_sel       = PWM_TIMER,
        .duty            = 0,
        .hpoint          = 0
    };
    ledc_channel_config(&ch2);
}

```

Figure 28: Functions `motor_forwards()`, `motor_reverse()`, `motor_stop()`

```

// Driver motor forward
void motor_forward(uint8_t speed) {
    ledc_set_duty(PWM_MODE, PWM_CHANNEL_1, speed);
    ledc_update_duty(PWM_MODE, PWM_CHANNEL_1);

    ledc_set_duty(PWM_MODE, PWM_CHANNEL_2, 0);
    ledc_update_duty(PWM_MODE, PWM_CHANNEL_2);
}

// Driver motor backwards
void motor_reverse(uint8_t speed) {
    ledc_set_duty(PWM_MODE, PWM_CHANNEL_1, 0);
    ledc_update_duty(PWM_MODE, PWM_CHANNEL_1);

    ledc_set_duty(PWM_MODE, PWM_CHANNEL_2, speed);
    ledc_update_duty(PWM_MODE, PWM_CHANNEL_2);
}

// Driver motor to a stop
void motor_stop(void) {
    ledc_set_duty(PWM_MODE, PWM_CHANNEL_1, 0);
    ledc_update_duty(PWM_MODE, PWM_CHANNEL_1);

    ledc_set_duty(PWM_MODE, PWM_CHANNEL_2, 0);
    ledc_update_duty(PWM_MODE, PWM_CHANNEL_2);
}

```

Figure 29: Function `ultrasonic_init()`

```

void ultrasonic_init() {
    gpio_set_direction(TRIG_GPIO, GPIO_MODE_OUTPUT);
    gpio_set_direction(ECHO_GPIO, GPIO_MODE_INPUT);

    gpio_set_level(TRIG_GPIO, 0);
}

```

Figure 30: Function `get_distance()`

```

float get_distance()
{
    for (int attempt = 0; attempt < 3; attempt++) // retry 3 times
    {
        int64_t start_time = 0;
        int64_t end_time = 0;
        int64_t timeout;

        // Trigger pulse
        gpio_set_level(TRIG_GPIO, 1);
        esp_rom_delay_us(10);
        gpio_set_level(TRIG_GPIO, 0);

        // Wait for echo HIGH
        timeout = esp_timer_get_time();
        while (gpio_get_level(ECHO_GPIO) == 0)
        {
            if (esp_timer_get_time() - timeout > 30000)
                break;
        }

        if (gpio_get_level(ECHO_GPIO) == 0)
            continue; // retry

        start_time = esp_timer_get_time();

        // Wait for echo LOW
        timeout = esp_timer_get_time();
        while (gpio_get_level(ECHO_GPIO) == 1)
        {
            if (esp_timer_get_time() - timeout > 30000)
                break;
        }

        if (gpio_get_level(ECHO_GPIO) == 1)
            continue; // retry

        end_time = esp_timer_get_time();

        int64_t duration = end_time - start_time;
        float distance = duration / 58.0;

        // Reject garbage values
        if (distance > 2 && distance < 400)
            return distance;
    }

    return -1; // only if all retries fail
}

```

Figure 31: Function send_distance()

```

void send_distance(float distance) {
    esp_err_t result = esp_now_send(controller_mac, (uint8_t *)&distance, sizeof(distance));

    if (result != ESP_OK) {
        ESP_LOGE(TAG, "Distance send failed");
    }
}

```

Figure 32: Function ultrasonic_task()

```

void ultrasonic_task(void *arg)
{
    float last_valid = 50;    // initial guess
    float filtered = 50;

    while(1)
    {
        float distance = get_distance();

        if (distance > 0)
        {
            // filter
            filtered = 0.7 * filtered + 0.3 * distance;
            last_valid = filtered;
        }
        else
        {
            // Use last valid value instead of -1
            distance = last_valid;
        }

        printf("Distance: %.2f cm\n", distance);
        send_distance(distance);

        vTaskDelay(pdMS_TO_TICKS(100));
    }
}

```

Appendix C – 3D CAD Photos

Figure 33: Rack and Pinion steering system

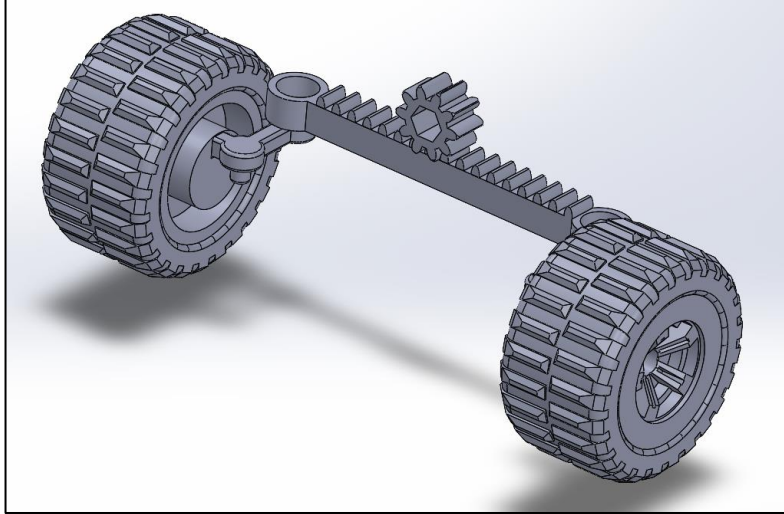


Figure 34: Top side view of the steering system integrated in the chassis

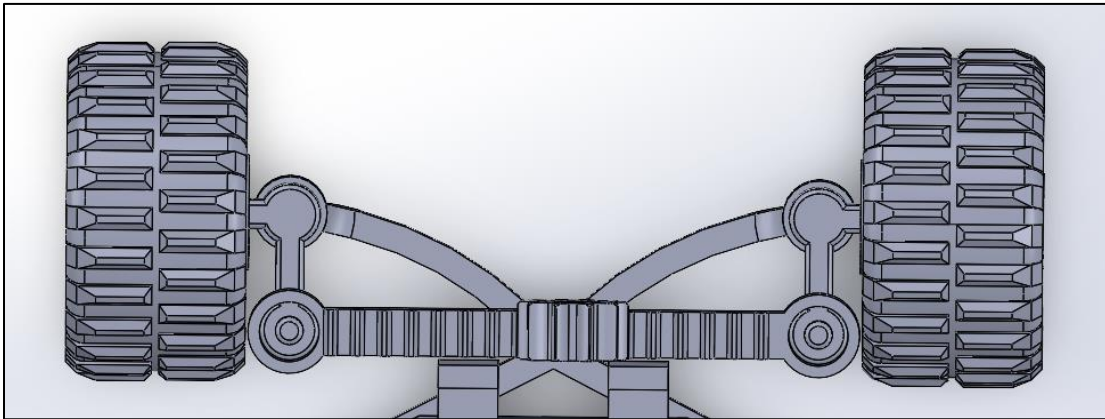


Figure 35: Angled view of the chassis

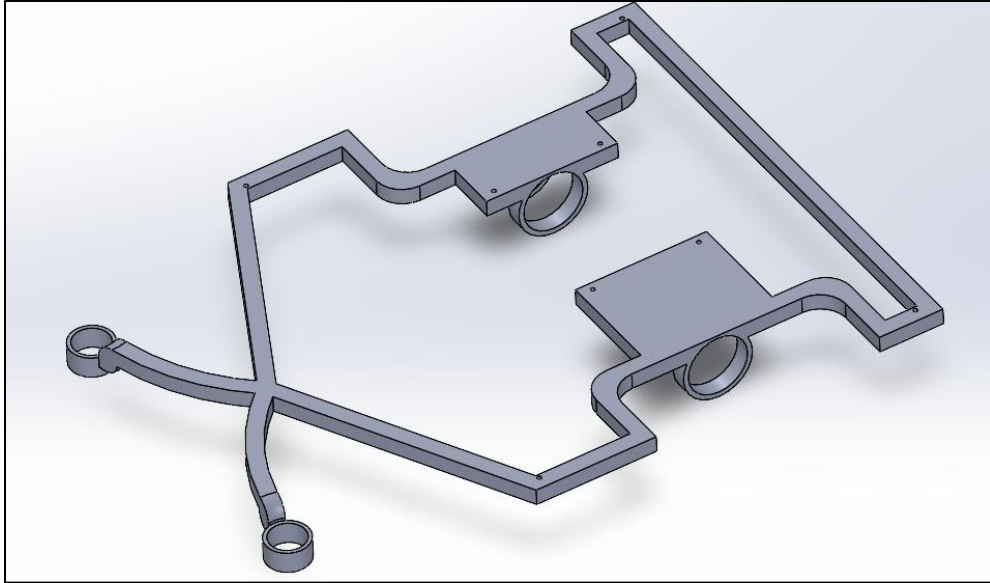


Figure 36: Bottom view of the chassis

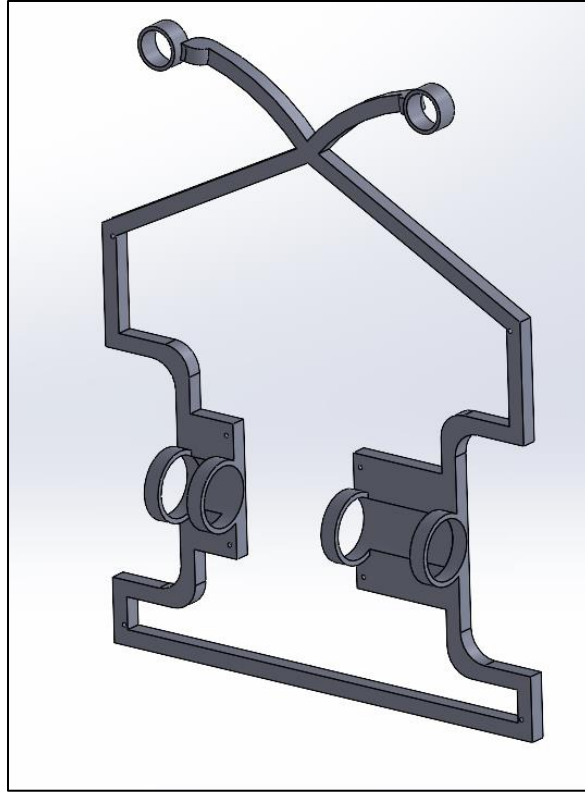


Figure 37: Differential system (not mounted)

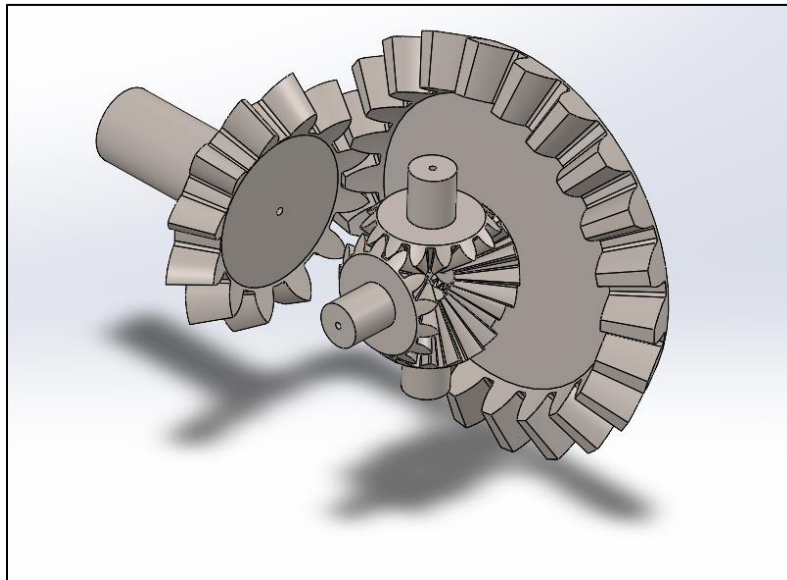


Figure 38: Crown gear finalized

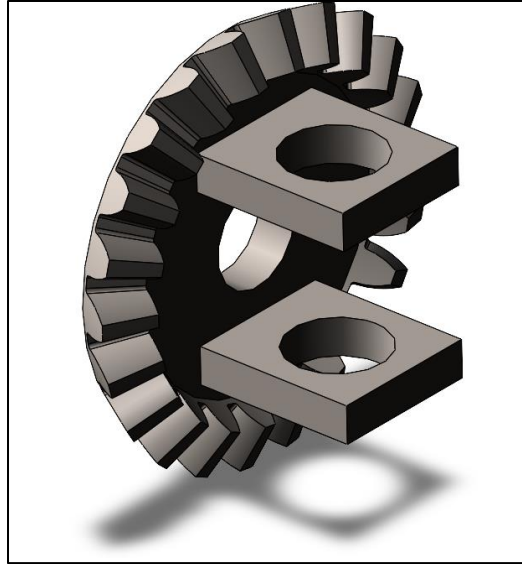


Figure 39: Differential system (mounted)

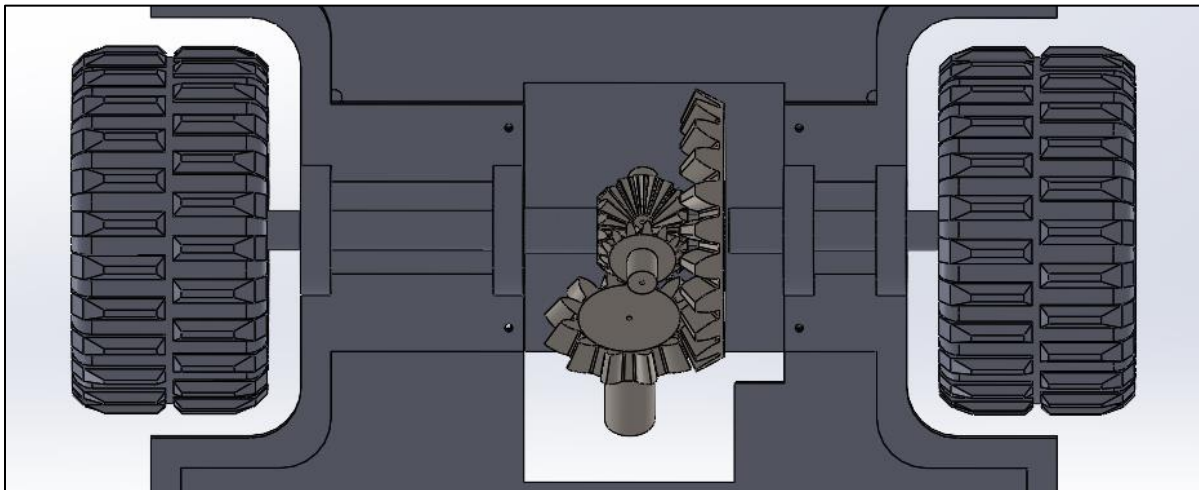


Figure 40: Car floor, battery and motor holder

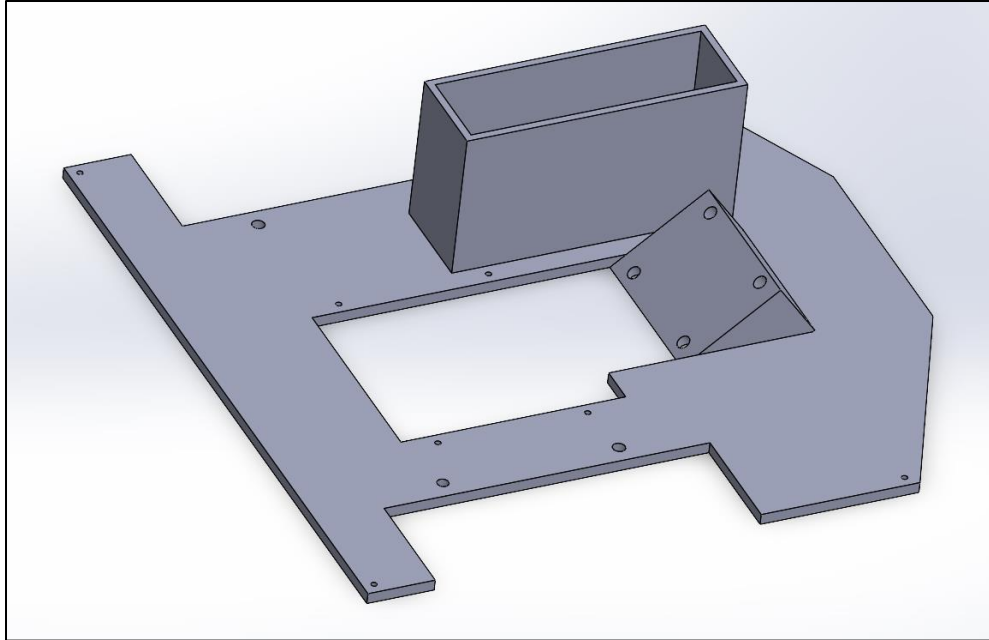
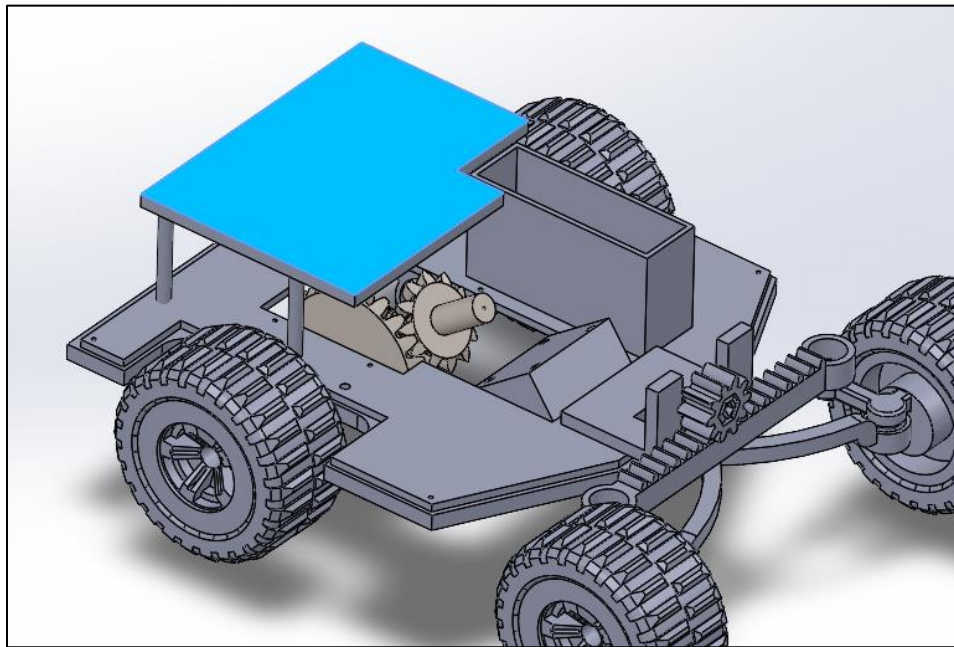


Figure 41: Fully assembled floor into chassis car design



The blue selection shows where the PCB goes

Figure 42: Metal Detection cable holder

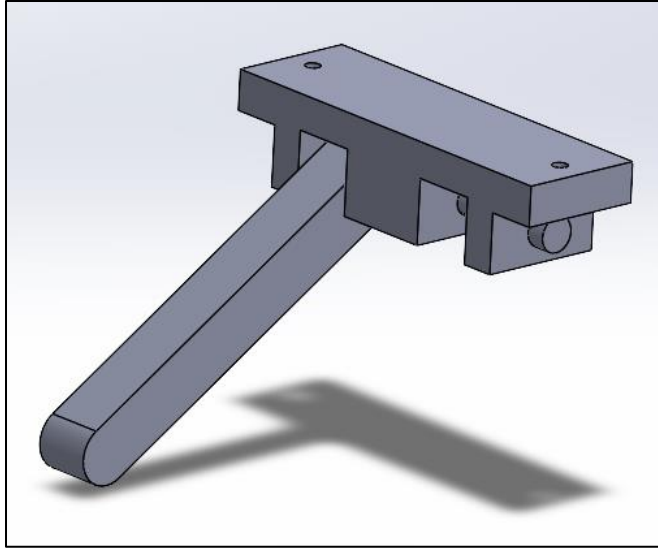


Figure 43: Front of the car

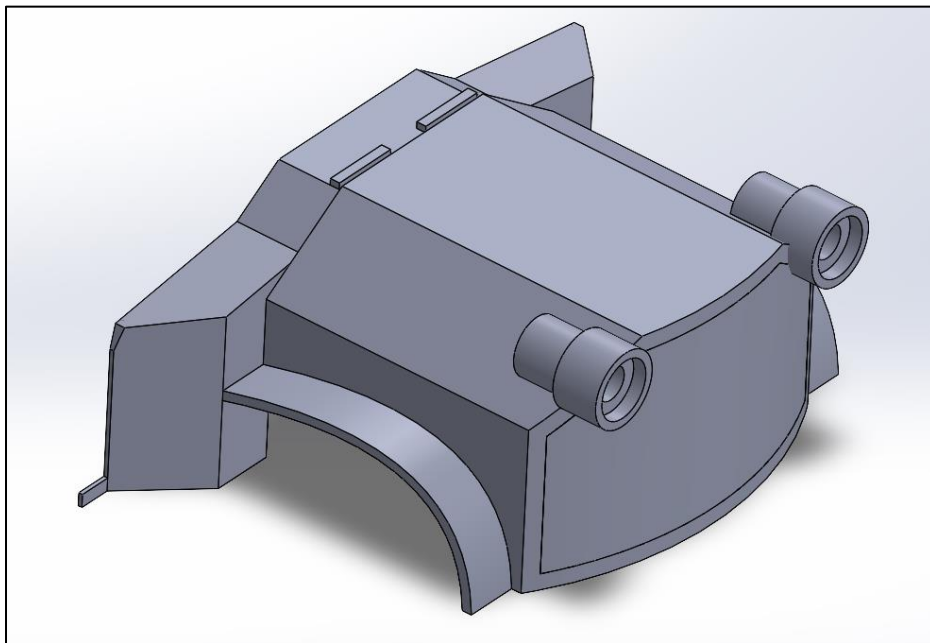


Figure 44: Back of the car

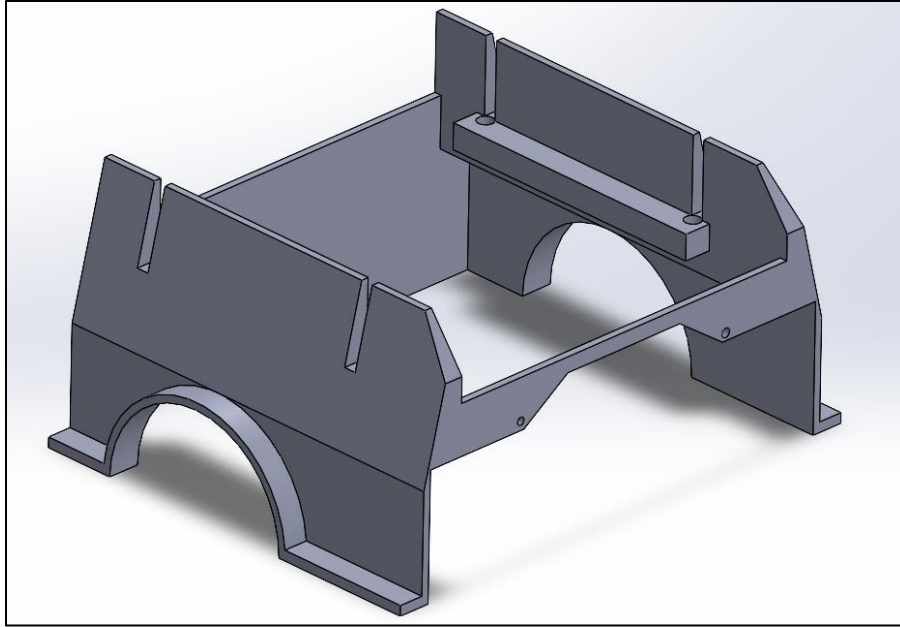


Figure 45: Cover for the Back of the Car Assembled

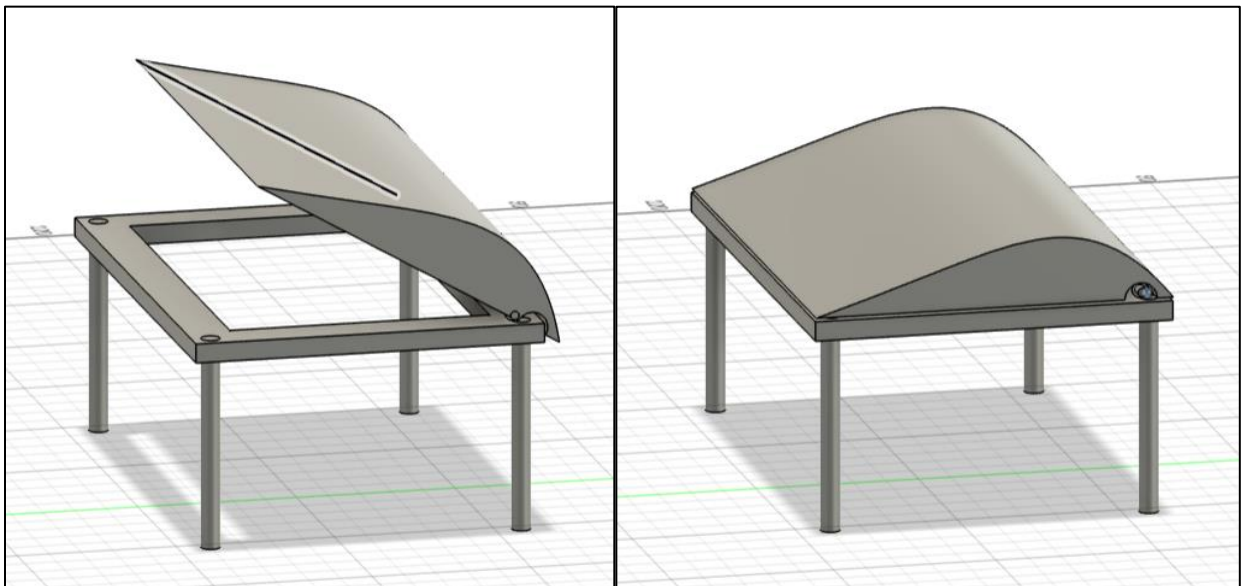


Figure 46: Steering Wheel Assembly

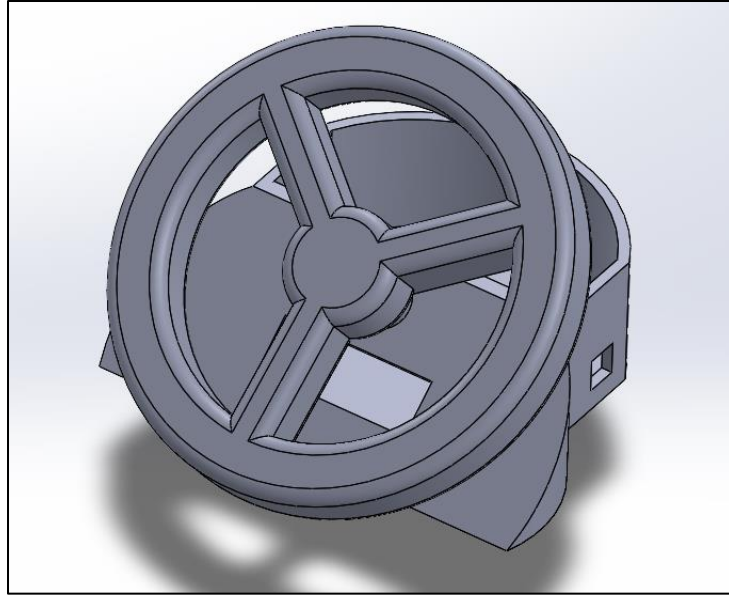


Figure 47: Potentiometer holder

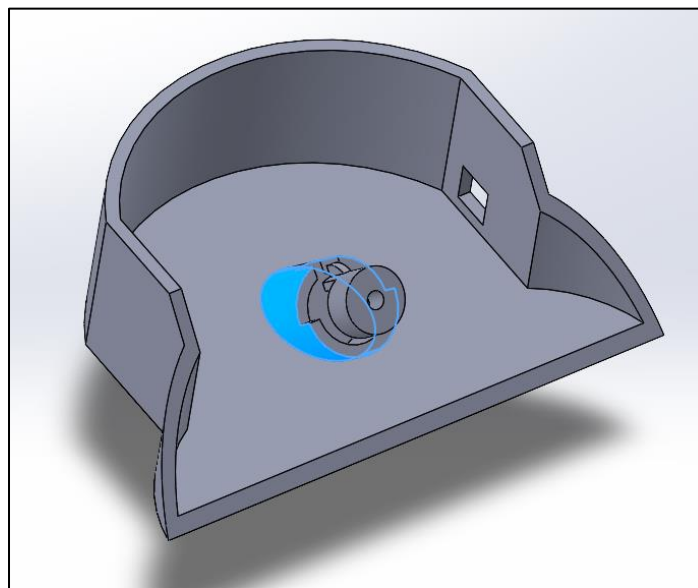


Figure 48: Button enclosure

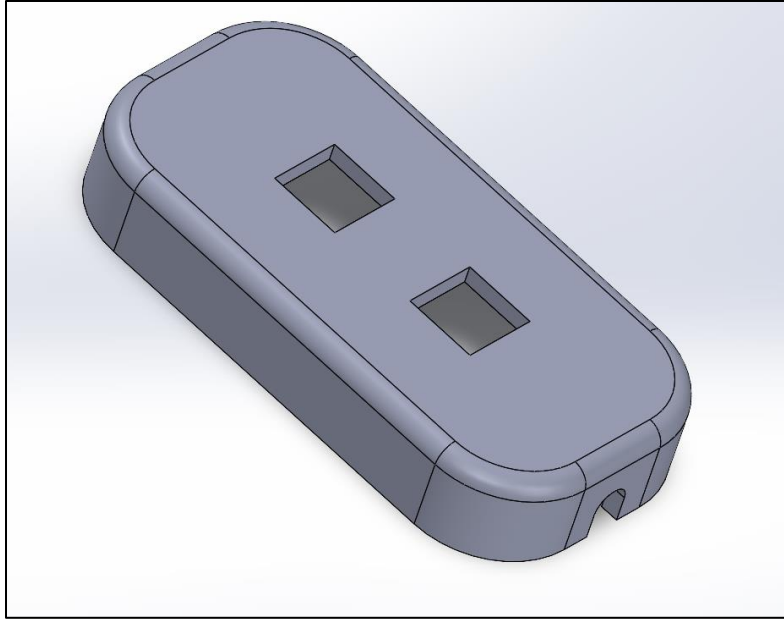


Figure 49: Bottom view of button holder

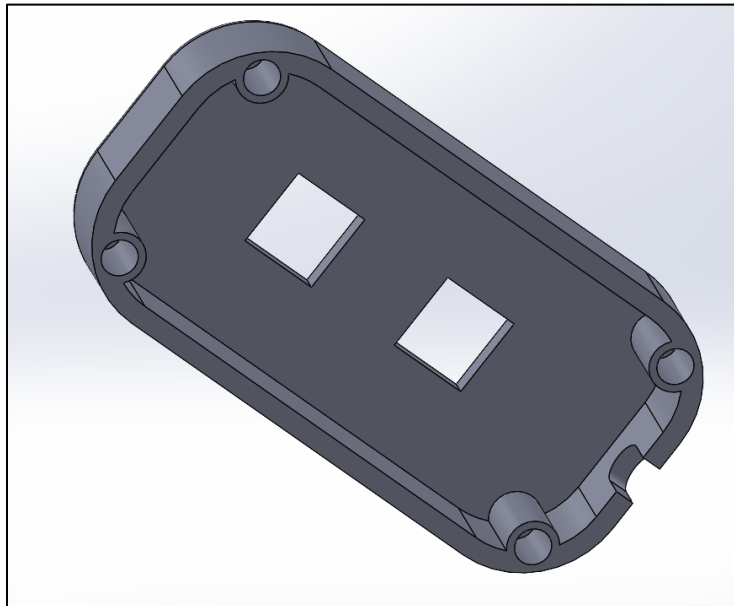


Figure 50: Bottom piece of button holder

